

Chapter 2

Learning Python for Artificial Intelligence

As we have seen how artificial Intelligence can be useful in agriculture but the next question always come in mind is that from where and how we can start to learn and what are the model of learning an artificial Intelligence. As Artificial Intelligence is a huge part of Computer Science which means we need an language as medium to perform it and we always prefer Python for that which is very dynamic and easy to learn. In this chapter we will learn about python basics so that in further implementation of artificial Intelligence it will be helpful to us.

I. Introduction

Python is a remarkable language that can claim to be easy and strong. You will be amazed to see how simple it is not to focus on the language you are programming, but on solving the problem. Guido van Rossum was the creator of the Python language which is named after the BBC show “Monty Python’s flying Circus”.

Python is formally introduced as:

“Python is an easy to learn, powerful programming language. It has efficient high-level data structures and a simple but effective approach to object-oriented programming. Python’s elegant syntax and dynamic typing, together with its interpreted nature, make it an ideal language for scripting and rapid application development in many areas on most platforms.”

II. Features f Python

- **Simple:** The Python language is simple and minimalist. Reading a good Python program almost feels like reading English, but it’s really tight! Python is one of its biggest assets, its pseudo-code existence. This helps you to focus instead of the language itself on the problematic solution.
- **Easy to Learn:** As you’ll see, Python can be started very quickly. As already described, Python has an extremely simple syntax.
- **Free and open source:** A FLOSS (Free and Open Source Software) example is Python. You can share free copies, read the source code of the software, make significant changes, and use some of it in free new programs. FLOSS is based on an intelligence-sharing network idea. Which is one of the factors Why Python is developed and is continually changed.
- **High Level Language:** You never have to bother about small information such as the management of memory use in your program when you make programs in Python, etc.

- **Portable:** Python has been adapted to (i.e., updated to) several sites because of its existence as an open source. Either of these systems will operate in all your Python programs without any modifications if you are diligent enough to avoid system-dependent functions. Python can also be used for GNU/Linux, Windows, FreeBSD, Macintosh, Solaris, OS/2, Amiga, AROS, AS/400, BeOS, OS/390, z/OS, Palm OS, Windows CE and PocketPC.

You can also make games for your machine, iPhone, iPad or Android with a platform like Kivy.

- **Extensible:** You could code some part of a program in c or c++ and then use it from a program called your Python if you need a vital part of code to run very quickly or you want any algorithm not to be opened.
- **Embeddable:** To offer users scripting capabilities for your software, you can incorporate Python in your C/C++ programs.
- **Interpreted:** A compiled program such as C or C++ is translated from the source language, i.e., C or C++ in a programming language (binary code i.e., 0s and 1s) with a compiler containing different flags and options The Linker/Loader software copies and begins running this program from hard disk to memory. On the other hand, Python does not require binary compilation. The software is run by the source code directly. Python dynamically transforms the source code into an intermediate type called bytecodes, then translates it into the computer's native language and then executes it. In fact, it's much easier to use Python because you don't have to worry if you compile the software to ensure the correct libraries are attached and loaded, etc. This makes your Python program more portable, as you can only copy your Python to a different machine and it just functions!
- **Object Oriented:** Python facilitates procedure-oriented programming as well as object-oriented programming. In procedure-oriented languages, the software is constructed around procedures or functions which are nothing but generic pieces of programs. The software is designed around objects in object-oriented languages that combine data and functionality. In comparative with major languages such as C++ or Java, Python has a good but simplified way to do OOP.
- **Extensive libraries:** The basic library of Python is immense. It will help you make various things with regular expressions, document creation, unit analysis, threading, data bases, web browsers, CGI, FTP, email, XML, XML-RPC, HTML, WAV, encryption, GUI and other system-dependent objects. All this should be considered anywhere Python is installed. This is known as the Theory of Python, including batteries.

Besides the main library, the Python Package Index also provides many other high-quality libraries.

III. Installation

A large number of platforms provide Python distribution. The binary code for your application must only be downloaded and Python enabled.

You need a C compiler to compile the source code by hand if the binary code of your platform is not usable. In order to make the choice of features required during your installation, you can compile the source code more flexibly.

A short picture of Python installation across different platforms is given here –

- **Unix and Linux Installation**

- Follow these steps to install Python on Unix/Linux machine.
- Open a Web browser and go to <https://www.python.org/downloads/>.
- Follow the link to download zipped source code available for Unix/Linux.
- Download and extract files.
- Editing the Modules/Setup file if you want to customize some options.
- `run./configure` script
- `make`
- `make install`
- This installs Python at the standard location `/usr/local/bin` and its libraries at `/usr/local/lib/pythonXX` where XX is the version of Python.

- **Windows Installation**

- Visit <https://www.python.org/downloads/> and download the latest version. At the time of this writing, it was Python 3.7 or greater version. The installation is just like any other Windows-based software.
- Note that if your Windows version is pre-Vista, you should download Python 3.4 only as later versions require newer versions of Windows.
- CAUTION: Make sure you check option Add Python 3.7 or greater version to PATH.
- To change install location, click on Customize installation, then Next and enter `C:\python35` (or another appropriate location) as the install location.
- If you didn't check the Add Python 3.7 or greater version PATH option earlier, check Add Python to environment variables. This does the same thing as Add Python 3.7 or greater version to PATH on the first install screen.
- You can choose to install Launcher for all users or not, it does not matter much. Launcher is used to switch between different versions of Python installed.
- If your path was not set correctly (by checking the Add Python 3.7 or greater version Path or Add Python to environment variables options), then follow the steps in the next section (DOS Prompt) to fix it. Otherwise, go to the Running Python prompt on Windows section in this document.

Note: For people who already know programming, if you are familiar with Docker, check out Python in Docker and Docker on Windows.

• Mac OS Installation

If you are on Mac OS X, it is recommended that you use Home brew to install Python 3. It is a great package installer for Mac OS X and it is really easy to use. If you don't have

Homebrew, you can install it using the following command:

```
$ ruby -e "$(curl -fsSL
```

```
https://raw.githubusercontent.com/Homebrew/install/master/install)"
```

We can update the package manager with the command below:

```
$ brew update
```

Now run the following command to install Python3 on your system:

```
$ brew install python3
```

IV. Setting up PATH

Programs and other executable files should be in a variety of directories to provide the operating systems with a search path listing the directories that the OS is searching for.

The route is stored as a named string maintained by the operating system in an environment variable. The command Shell and other programs are supplied with this variable.

For Unix and Windows, the path attribute is called PATH (Unix is case-sensitive; Windows does not).

The installer manages the track information in Mac OS. You must add a Python directory to your path to invoke the Python interpreter from any specific directory.

• Setting Path at Unix/Linux

To add the Python directory to the path for a particular session in Unix –

- In the csh shell – type `setenv PATH "$PATH:/usr/local/bin/python"` and press Enter.
- In the bash shell (Linux) – type `export PATH="$PATH:/usr/local/bin/python"` and press Enter.
- In the sh or ksh shell – type `PATH="$PATH:/usr/local/bin/python"` and press Enter.

Note: /usr/local/bin/python is the path of the Python directory.

• Setting Path at Windows

To add the Python directory to the path for a particular session in Windows –

At the command prompt –

```
type path %path%;C:\Python and press Enter.
```

Note : C:\Python is the path of the Python directory.

V. Running Python with GUI

If you have a GUI on your machine to support Python, you can run Python from a GUI environment as well.

- **Unix:** IDLE is the very first Unix IDE for Python
- **Windows:** Python Win is the first Windows interface for Python and is an IDE with a GUI.
- **Macintosh:** The Macintosh version of Python along with the IDLE IDE is available from the main website, downloadable as either Mac Binary or Bin Hex'd files.

You should take support from your system manager if you are unable to set up the framework properly. Ensure sure that the Python system is properly designed and runs well.

Another site called Anaconda can be used. We are also open. It includes hundreds of common data science packages, the conda package and the Windows, Linux and MacOS virtual environment manager. You can access it from the <https://www.anaconda.com/download/> link using your operating system.

In this book all the programs use Python 3.6 on MS Windows 10.

VI. Getting Started with python

(a) First Step:

Now in Python we're going to see how to run a standard 'Hello World.' You'll learn how Python programs are written, saved and run.

There are two methods of using the interactive interpreter prompt or using a source file with Python to run the software. let's now see how these two approaches are used.

By the Interpreter Prompt:

Open the terminal in your operating system and then open the Python prompt with the python3 key. The [enter] key will be pressed and open.

You will see >> when you start typing stuff once you started Python. This is known as the python interpreter prompt.

At the Python interpreter prompt, type:

```
>>print ("Hello World")
```

Then press the [Enter] key and as an output it will give-> Hello World.

Here is an example how screen will look in Fig 2.1

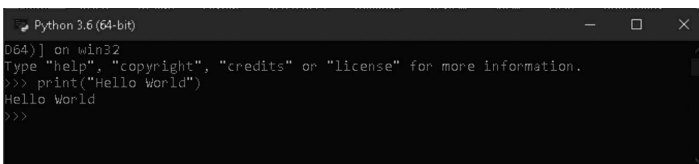
A screenshot of a terminal window titled "Python 3.6 (64-bit)". The window shows the following text: "064)] on win32", "Type 'help', 'copyright', 'credits' or 'license' for more information.", ">>> print('Hello World')", "Hello World", and ">>>". The terminal has a dark background and white text. The window has standard Windows window controls (minimize, maximize, close) in the top right corner.

Fig. 2.1 Python Example in IDE

Quitting the Interpreter Prompt

You can exit the interpreter by pressing [ctrl + d] or entering exit) (if you're using the GNU/Linux or OS X shell (note: remember to use parentheses,))), (and then the key [enter].

On command prompt/ interpreter for Windows, press [ctrl+z] and then press [enter].

Using an Editor

A tool for writing and editing code is a code editor. Which are generally easy to know and can be fantastic. However, when your software is bigger, you will have to check and debug your code.

Your code is a lot better understanding than a text editor in the IDE (integrated developing environment). It usually includes features including construct automation, application layout, testing and debugging. It will improve your research considerably. The downside is that IDEs are difficult to use.

There are so many Editors like python IDE, Atom, PyCharm, Visual Studio code etc. These can be used for programming purpose and for development of various projects.

(b) Basics:

It's not enough just to print hello world, is it? How would like to do something else -like to make some changes, use them and get something out of them. In Python, by using constants and variables it can be done, and in this chapter, several other concepts will be discussed.

COMMENTS

Comments are a text to the right of the # symbol and are often useful for technology readers.

For example:

```
>>print ('hello world')# Note that print is a function
or:
# Note that print is a function
>>print ('hello world')
```

Use as many helpful comments as possible to clarify important decisions in your program to clarify important facts, explain problems you are trying to solve, explain problems you are trying to resolve, respectively.

Code shows you how, comments explains why.

LITERAL CONSTANTS

A number like 5, 1.23, or a string like 'This is a string' or 'It's a string,' an example of literal constant.

It is referred to as literal-you are simply using its meaning. The number 2 is always and is nothing else-it is a constant, because its value can't be modified. All of these are also called literal constants.

NUMBERS

Two types of numbers which are Integers and floats are commonly used.

One example is 2 that is just a number.

Examples of float numbers are 3.23 and 52.3E-4 (or shorter floats). The E notation displays 10 competencies. 52.3E-4 is in this case $52.3 * 10^{-4}$.

STRINGS

A string is a character set. In essence, strings are just a bunch of words.

Nearly every Python program you write will have strings used, so be very careful about the following section.

SINGLE QUOTE

You can specify strings using single quotes such as 'Quote me on this'.

All white space i.e., spaces and tabs, within the quotes, are preserved as-is.

DOUBLE QUOTE

Strings in double quotes work exactly the same way as strings in single quotes.

An example is "What's your name?"

TRIPLE QUOTE

You can specify multi-line strings using triple quotes - (""" or '''). You can use single quotes and double quotes freely within the triple quotes.

An example is:

```
'''This is a multi-line string. This is the first line.
```

```
This is the second line.
```

```
"What's your name?," I asked.
```

```
He said "Bond, James Bond."
```

```
'''
```

Strings are IMMUTABLE: This means you can't alter a string once you've created it. And if it may seem a negative thing, it isn't really. We can see how the different services that we see afterwards are not limited by this.

VARIABLES:

The only way to store and manipulate data would soon be to be repetitive with only literal constants. Variables enter the view here.

Variables are just what the name means-their values can differ, i.e., something can be stored with a variable. Variables are just part of the memory of your machine in which information is stored. You need a way to control these variables and then call them, as opposed to literal constants.

IDENTIFIERS

Variables are examples of identifiers. Identifiers are names given to identify something.

There are some rules you have to follow for naming identifiers:

- The first character of the identifier must be a letter of the alphabet (uppercase ASCII or lowercase ASCII or Unicode character) or an underscore (_).
- The rest of the identifier name can consist of letters (uppercase ASCII or lowercase ASCII or Unicode character), underscores (_) or digits (0-9).
- Identifier names are case-sensitive. For example, myname and myName are not the same. Note the lowercase n in the former and the uppercase N in the latter.
- Examples of valid identifier names are i, name_2_3. Examples of invalid identifier names are 2things, this is spaced out, my-name and >a1b2_c3.

DATA TYPES

Different types of variables can hold values called data types. The key forms, which we have described, are numbers and strings. We will discuss later chapters how our own classes are to be formed.

Python refers to anything used as an entity in a program. It is generically said. They say “the object,” rather than “the something,”

Python is highly object-oriented, in the sense that all is an entity, which includes numbers, strings and functions.

We will now see how variables with concrete constants can be used. Run the program and save the following example.

How to Write a program in python:

Steps to follow:

- Open editor/Interpreter
- Type the program code as in the example
- Save it as a file with a name with a [DOT]py or “.py” Extension
- Run the interpreter with an command *python filename.py* to run the program or you can use ru button on an editor to run the program.

Note: Filename can be anything but it should have an.py extension

Example: Type and run the following program

```
>>> # Filename : var.py
i = 5
print (i)
i = i + 1
print (i)
s = '''This is a multi-line string.
This is the second line
Welcome to artificial intelligence.'''
print (s)
```


Output:

5

6

This is a multi-line string.

This is the second line

Welcome to artificial Intelligence.

How It Works

Here's how this program works. First, we assign the literal constant value 5 to the variable `i` using the assignment operator (`=`). This line is called a statement because it states that something should be done and in this case, we connect the variable name `i` to the value 5. Next, we print the value of `i` using the print statement which, unsurprisingly, just prints the value of the variable to the screen.

Then we add 1 to the value stored in `i` and store it back. We then print it and expectedly, we get the value 6.

Similarly, we assign the literal string to the variable `s` and then print it.

INDENTATION

Whitespace in Python is significant. White space is actually necessary at the start of the line. That is what is known as indentation. At the beginning of the logical line, the leading whitespace, or tabs, is used to decide the indentation level of the logical line that is used in effect to define the aggregation of statements.

That implies all statements that are in general will have the same indentation. Both these statements are referred to as a block. Throughout the following pages, we can see examples of how blocks matter.

One thing you should know is that misplacement will lead to errors in the program. For instance:

```
>>> i = 5
# Error below! Notice a single space at the start of the line
print ('Value is', i)
print ('I repeat, the value is', i)
```

You get the following error while running this:

```
File "whitespace.py", line 3
```

```
print ('Value is', i)
```

```
^
```

```
Indentation Error: unexpected indent
```

Note at the beginning of the second line that there is a single space. Python's error means that the program's syntax is not accurate, and that the program has not been written correctly. It ensures for you that new blocks of statements cannot be started randomly (except for the existing principal block you've used all along).

OPERATORS & EXPRESSIONS

The majority of statements you write (logical lines) will contain words. An expression $2 + 3$ is a simple example. An expression can be divided into operators and operands.

Operators are functions that can do anything, symbols like $+$ or special keys are symbolic of them. Operators need other data and those data are referred to as operands. 2 and 3 are technicians in this case

We will discuss the operators and their use briefly.

Note that the expressions in the examples can be tested interactively with the interpreter. For instance, the interactive Python interpreter prompt is used to check expression $7 + 5$:

```
>>>7 + 5
12
>>>3 * 8
24
>>>
```

Here is a short rundown of the available operators:

- $+$ (plus)

Adds two objects

$+ 5$ gives 8. $'a' + 'b'$ gives $'ab'$
- $-$ (minus)

Gives the subtraction of one number from the other; if the first operand is absent it is assumed to be zero.

-5.2 gives a negative number and $50 - 24$ gives 26
- $*$ (multiply)

Gives the multiplication of the two numbers or returns the string repeated that many times.

$* 3$ gives 6. $'la' * 3$ gives $'lalala'$
- $**$ (power)

Returns x to the power of y

$** 4$ gives 81 (i.e., $3 * 3 * 3 * 3$)
- $/$ (divide)

Divide x by y

3 gives 4.3333333333333333
- $//$ (divide and floor)

Divide x by y and round the answer down to the nearest integer value. Note that if one of the values is a float, you'll get back a float.

$13 // 3$ gives 4

-13 // 3 gives -5

9//1.81 gives 4.0

- % (modulo)

Returns the remainder of the division

3 gives 1. -25.5 % 2.25 gives 1.5.

- << (left shift)

Shifts the bits of the number to the left by the number of bits specified. (Each number is represented in memory by bits or binary digits i.e., 0 and 1)

2 << 2 gives 8. 2 is represented by 10 in bits.

Left shifting by 2 bits gives 1000 which represents the decimal 8.

- >> (right shift)

Shifts the bits of the number to the right by the number of bits specified.

11 >> 1 gives 5.

11 is represented in bits by 1011 which when right shifted by 1 bit gives which is the decimal 5.

- & (bit-wise AND)

Bit-wise AND of the numbers

5 & 3 gives 1.

- | (bit-wise OR)

Bitwise OR of the numbers

5 | 3 gives 7

- ^ (bit-wise XOR)

Bitwise XOR of the numbers

^ 3 gives 6

- ~ (bit-wise invert)

The bit-wise inversion of x is - (x+1)

~5 gives -6.

- < (less than)

Returns whether x is less than y. All comparison operators return True or False.

Note the capitalization of these names.

5 < 3 gives False and 3 < 5 gives True.

Comparisons can be chained arbitrarily: 3 < 5 < 7 gives True.

- > (greater than)

Returns whether x is greater than y

5 > 3 returns True. If both operands are numbers, they are first converted to a common type. Otherwise, it always returns False.

`<=` (less than or equal to)

Returns whether x is less than or equal to y

`x = 3; y = 6; x <= y` returns True

`>=` (greater than or equal to)

Returns whether x is greater than or equal to y

`x = 4; y = 3; x >= 3` returns True

- `==` (equal to)

Compares if the objects are equal

`x = 2; y = 2; x == y` returns True

`x = 'str'; y = 'stR'; x == y` returns False

`x = 'str'; y = 'str'; x == y` returns True

- `!=` (not equal to)

Compares if the objects are not equal

`x = 2; y = 3; x != y` returns True

- `not` (boolean NOT)

If x is True, it returns False. If x is False, it returns True.

`x = True; not x` returns False.

- `and` (boolean AND)

x and y return False if x is False, else it returns evaluation of y

`x = False; y = True; x and y` return False since x is False. In this case, Python will not evaluate y since it knows that the left-hand side of the 'and' expression is

False which implies that the whole expression will be False irrespective of the other values. This is called short-circuit evaluation.

- `or` (boolean OR)

If x is True, it returns True, else it returns evaluation of y

`x = True; y = False; x or y` returns True. Short-circuit evaluation applies here as well

CONTROL FLOW

In the programs that we've seen until now, Python's exact top-down statements also were faithfully implemented. How if the rhythm of how it functions was to change? Do you, for example, like printing 'Good morning' or 'Good evening' depending on the daytime, want the system to take some decisions and do certain things according to the various situations?

As you might have imagined, the control flow statements are used to accomplish this. Within Python-if for and While-there are three statements of control flow.

- If Statement

If the condition is true, we run a block (known as the if-block), otherwise we must process a block (known as the other block). If the condition is true, we must do that. This will be done. The other element is optional.

Example (save as if.py):

```
>>>number = 77
    guess = int (input ('Enter an integer : '))
    if guess == number:
        # New block starts here
        print ('Congratulations, you guessed it.')
        print (' (but you do not win any prizes!')
        # New block ends here
elif guess < number:
    # Another block, in this we can use if for n number of times.
    print ('No, it is a little higher than that')
    # You can do whatever you want in a block...
    else:
print ('No, it is a little lower than that')
    # you must have guessed > number to reach here
print ('Done')
    # This last statement is always executed,
    # after the if statement is executed.:
```

OUTPUT:

```
$ python if.py
Enter an integer: 22
No, it is a little higher than that
Done
```

We take guesses from the user in this program and check that it is the number we contain.

We have specified the number of the variable to any integer, say 23. We then use the input function to guess the user. Which are Reusable pieces of programs are just featuring. We provide a string to the integrated input feature to print on the screen and wait for user input. The input() function returns what we have entered as a string after we type something and click the kbd:[enter] key. This string is translated into an integer with int and stored in the conjecture variable. In general, the int is a class, but all that you have to learn right now is that you can use it to convert a string to an integer.

Next, we equate the user's guess to our selected number. by printing a success message if they are equal. Note, to say Python what statements to which block are used in indentation stages. In Python, indentation is therefore so necessary.

Note how the statement if at the bottom contains a column – we inform Python that it follows a block of statements.

Then we test if the assumption is less than the number, and if so, we will tell the consumer a bit higher than the assumptions. The `elif` clause that have been used here simply incorporates two similar if not other claims into a single if-elif else declaration. It improves the system and reduces the amount of indentation required.

At the end of the logic section, `elif` and other statements must always have a colon followed by a series of statements (which of course is properly indented).

In the if-block of an if statement and so on, you can have another if statement-it's called a nested if statement.

Remember that the `elif` and `else` parts are optional. A minimal valid if statement is:

```
>>> if True:
    print ('Yes, it is true')
```

Upon Python completes the complete if declaration along with the corresponding `elif` and other clauses, it moves on to the next statement in the if declaration sequence. The next statement is the `print ('Done')` statement. This is a big block (where execution of the program starts). Then Python sees and finishes the end of the plan.

- For Statement

The statement `for .. in` is another blowing statement, which passes through each element in a sequence through a sequence of items. In the following sections we will see more about series. Now you just have to know that a series is just a sequence of objects.

Example (save as `for.py`):

```
for i in range (1, 5):
    print (i)
else:
    print ('The for loop is over')
```

Output:

```
$ python for.py
1
2
3
4
The for loop is over
```

We print a sequence of numbers in this program. We use the built-in `range` function to produce this number series. All we do here is supply it with two numbers and return a series of numbers from the first to the second number. `Range (1,5)` for example offers the `[1, 2, 3, 4]` sequence. The `range` takes by default a move number of 1. The phase number is supplied if we supply a third number in the sequence. `Range (1,5,2)`, for

example, lists [1,3]. Notice that the range extends up to the second number i.e., the second number is not included.

Remember that, if you want a complete number set, for example, the list (range (5)) would result in [0,1, 2, 3, 4] list (s) is explained in the data structure chapter of the data structures class. The list (s) is then iterated for the loop-for the I in the field (1,5), the list (s) in [1, 2, 3, 4] is equal to the assignment of each number (s) in the series In this case, the value is only printed in the statement row.

Notice that the other segment is optional. When used, it is only done only after the loop has finished, unless a break statement is found. Note that for every sequence the.. in loop works. Here, we have a list of numbers created by the integrated range function, but we can generally use any type of object sequence!.

- **While Statement**

The while statement lets you execute a series of statements repeatedly, given that a condition is valid. An example of a so-called looping statement is a certain statement. An optional clause can be included in a certain sentence.

Example (save as while.py):

```
number = 23
running = True
while running:
    guess = int (input ('Enter an integer : '))
        if guess == number:
    print ('Congratulations, you guessed it.')
        # this causes the while loop to stop
    running = False
elif guess < number:
    print ('No, it is a little higher than that.')
else:
    print ('No, it is a little lower than that.')
else:
    print ('The while loop is over.')
# Do anything else you want to do here
print ('Done')
```

Output:

```
$ python while.py
Enter an integer : 50
No, it is a little lower than that.
Enter an integer : 22
No, it is a little higher than that.
Enter an integer : 23
```

```
Congratulations, you guessed it.  
The while loop is over.  
Done
```

We are still playing the devination game in this program, but the benefit is that the user can only deviate until he thinks right, as we did in the last segment-there is no need to run it repeatedly. This shows the use of the While Statement properly.

We shift the input to and from the while loop and set the variable to True before the while loop. First, we test if the variable is true and then proceed to execute it during the process. After this block has been completed, the condition is tested again which is the running variable in this case. If it is valid, we will again run the While block, otherwise the optional other block will continue to be executed, so the following sentence will follow.

If the condition is incorrect the other block is executed, even though the condition is verified for the first time. If there's another clause for a certain loop, it will only work if you don't break out of the loop.

Truth and False are called Boolean forms and can be seen as being similar to 1 and 0.

THE BREAK STATEMENT

The break statement is used to break out of a loop statement i.e., avoid a loop declaration, even though it has not become false or the item sequence is not completely iterated.

A significant point is that every accompanying loop block is not carried out when you break off a for or during loop.

Example (save as break.py):

```
while True:  
    s = input ('Enter something : ')  
    if s == 'quit':  
        break  
    print ('Length of the string is', len (s))  
print ('Done')
```

Output:

```
$ python break.py  
Enter something : Programming is fun  
Length of the string is 18  
Enter something : When the work is done  
Length of the string is 21  
Enter something : if you wanna make your work also fun:
```



```
Length of the string is 37
Enter something : use Python!
Length of the string is 11
Enter something : quit
Done
```

THE CONTINUE STATEMENT

Using the continuation rule, Python will save the rest of the statements in the current loop block and start with the next iteration of the loop.

Example (save as continue.py):

```
while True:
    s = input ('Enter something : ')
    if s == 'quit':
        break
    if len (s) < 3:
        print ('Too small')
        continue
    print ('Input is of sufficient length')
# Do other kinds of processing here...
```

Output:

```
$ python continue.py
Enter something : a
Too small
Enter something : 12
Too small
Enter something : abc
Input is of sufficient length
Enter something : quit
```

FUNCTIONS

Reusable program pieces are functions. You can assign a name to a block of statements, which you can use anywhere and any number of times to run the block using the defined name in your program. It is called function calling. Several integrated features like len and range have already been used.

Possibly the main building block for any untrivial applications (in any language of programming). Using the def keyword, functions are specified. After this keyword, a function name follows a couple of parentheses that may include the names of variables

and the final column ending the line. The set of declarations that are part of this feature will follow next. An example shows that it's really simple:

Example (save as function1.py):

```
def say_hello ():
    # block belonging to the function
    print ('Welcome to agriculture')
    # End of function

say_hello () # call the function
say_hello () # call the function again
```

Output:

```
$ python function1.py
Welcome to agriculture
Welcome to agriculture
```

LOCAL VARIABLES

When you declare variables inside a function definition, they are not related in any way to other variables with the same names used outside the function - i.e., variable names are local to the function. This is called the scope of the variable. All variables have the scope of the block they are declared in starting from the point of definition of the name.

Example (save as function_local.py):

```
x = 50
def func (x):
    print ('x is', x)
    x = 2
    print ('Changed local x to', x)

func (x)
print ('x is still', x)
```

Output:

```
$ python function_local.py
x is 50
Changed local x to 2
x is still 50
```

THE GLOBAL STATEMENT

You must tell Python that a given name is not local, but global if you are to assign a meaning to a name at the top level of the program (i.e., not within any scope, such as

the functions or classes). We use the global statement to do this. Without the global statement, a value can not be assigned to a variable specified outside of a function.

You may use values specified outside of the feature. This is not recommended and should be avoided because the user of the program does not know where the meaning of this variable is. It is amply clear from the global statement that the variable is specified in the external block.

Example (save as `function_global.py`):

```
x = 50
def func ():
    global x
    print ('x is', x)
    x = 2
    print ('Changed global x to', x)
func ()
print ('Value of x is', x)
```

Output:

```
$ python function_global.py
x is 50
Changed global x to 2
Value of x is 2
```

DEFAULT ARGUMENT VALUES

For certain features, if the user does not want to supply them with values, you can optionize for those parameters and use default values. The default argument values are used to do that. By adding parameter names to the parameter name of the assignment operator (=) specification, you can set the default parameter value for parameters.

Remember that the default statement is a constant value. More specifically-it is discussed in detail in later chapters-the default argument values must be immutable.

Example (save as `function_default.py`):

```
def say (message, times=1):
    print (message * times)
say ('Hello')
say ('World', 5)
```

Output:

```
$ python function_default.py
Hello
WorldWorldWorldWorldWorld
```

KEYWORD ARGUMENTS

The special syntax `**kwargs` in function definitions in python is used to pass a keyworded, variable-length argument list. We use the name `kwargs` with the double star. The reason is because the double star allows us to pass through keyword arguments (and any number of them).

A keyword argument is where you provide a name to the variable as you pass it into the function.

One can think of the `kwargs` as being a dictionary that maps each keyword to the value that we pass alongside it. That is why when we iterate over the `kwargs` there doesn't seem to be any order in which they were printed out.

Example (save as `function_keyword.py`):

```
def func (a, b=5, c=10):
    print ('a is', a, 'and b is', b, 'and c is', c)

func (3, 7)
func (25, c=24)
func (c=50, a=100)
```

Output:

```
$ python function_keyword.py
a is 3 and b is 7 and c is 10
a is 25 and b is 5 and c is 24
a is 100 and b is 5 and c is 50
```

VARARGS PARAMETERS

Sometimes you might want to define a function that can take any number of parameters, i.e., variable number of arguments, this can be achieved by using the stars (save as `function_varargs.py`):

```
def total (a=5, *numbers, **phonebook):
    print ('a', a)

    #iterate through all the items in tuple
    for single_item in numbers:
        print ('single_item', single_item)

    #iterate through all the items in dictionary
    for first_part, second_part in phonebook.items ():
        print (first_part,second_part)

    total (10,1,2,3,Jack=1123,John=2231,Inge=1560)
```

Output:

```
$ python function_varargs.py
a 10
```

```
single_item 1
single_item 2
single_item 3
Inge 1560
John 2231
Jack 1123
```

RETURN STATEMENT

The return statement is used to return from a function i.e., break out of the function. We can optionally return a value from the function as well.

Example (save as function_return.py):

```
def maximum (x, y):
    if x > y:
        return x
    elif x == y:
        return 'The numbers are equal'
    else:
        return y
print (maximum (2, 3))
```

Output:

```
$ python function_return.py
3
```

DATA STRUCTURES

Essentially, data structures are just that-structures that can keep all data together. In other words, a list of similar information is processed. There are in the Python list, tuple, dictionary and set four built-in data structures. We'll see how each of them can be used and how they improve our life.

- **LIST:** A list is the data structure that comprises an ordered set of objects, i.e., a series of objects can be stored in a list. You can imagine this easily, if you can think of a shopping list with a list of things to buy except that each item is actually put separately on your shopping list, while Python lists commas.

In square brackets the items collection should be included, so Python can understand that you define a collection. You can add, delete or check for the things in the list once you have built a list. we are able to add and remove items, a list is a mutable data type, i.e., it can be modified.

Example (save as list.py):

```
# This is my shopping list
shoplist = ['apple', 'mango', 'carrot', 'banana']
print ('I have', len (shoplist), 'items to purchase.')
```

```

print ('These items are:', end=' ')
for item in shoplist:
    print (item, end=' ')
print ('\nI also have to buy rice.')
shoplist.append ('rice')
print ('My shopping list is now', shoplist)
print ('I will sort my list now')
shoplist.sort ()
print ('Sorted shopping list is', shoplist)
print ('The first item I will buy is', shoplist[0])
olditem = shoplist[0]
del shoplist[0]
print ('I bought the', olditem)
print ('My shopping list is now', shoplist)

```

Output:

```

$ python ds_using_list.py
I have 4 items to purchase.
These items are: apple mango carrot banana
I also have to buy rice.
My shopping list is now ['apple', 'mango', 'carrot', 'banana',
                        'rice']

I will sort my list now
Sorted shopping list is ['apple', 'banana', 'carrot', 'mango',
                        'rice']

The first item I will buy is apple
I bought the apple
My shopping list is now ['banana', 'carrot', 'mango', 'rice']

```

- **TUPLE:** Tuples are used to hold several objects together. Think of them as lists, but without the broad features that you get from the list class. One of the most important characteristics of tuples is their unchangeable nature, that is, you can't alter tuples.

Tuples are identified in the optional parenthesis by indicating commas-split objects. Tuples are typically used where a statement or function specified by the user may assume safely that the set of values does not change (i.e., the multiple of the values used).

Example (save as tuple.py):

```

# I would recommend always using parentheses
# to indicate start and end of tuple
# even though parentheses are optional.

```

```
# Explicit is better than implicit.
zoo = ('python', 'elephant', 'penguin')
print ('Number of animals in the zoo is', len (zoo))
new_zoo = 'monkey', 'camel', zoo
    # parentheses not required but are a good idea
print ('Number of cages in the new zoo is', len (new_zoo))
print ('All animals in new zoo are', new_zoo)
print ('Animals brought from old zoo are', new_zoo[2])
print ('Last animal brought from old zoo is', new_zoo[2][2])
print ('Number of animals in the new zoo is', len (new_zoo)
                                             1+len (new_zoo[2]))
```

Output:

```
$ python ds_using_tuple.py
Number of animals in the zoo is 3
Number of cages in the new zoo is 3
All animals in new zoo are ('monkey', 'camel', ('python',
                                                'elephant', 'penguin'))
Animals brought from old zoo are ('python', 'elephant',
                                  'penguin')
Last animal brought from old zoo is penguin
Number of animals in the new zoo is 5
```

DICTIONARY

A dictionary is like an address-book in which you can find an individual's address or contact information only with the individual's name, that is, associating keys (name) with values (details). Please remember that the key must be special, such like if you have two people of the same name, you cannot find the correct keys.

You can only use immutable (such as strings) objects for dictionary keys, but for dictionary values you can use dynamic or moving objects. This simply means that only simple objects can be used for keys.

The dictionary defines keys and values pairs by notation

```
d = {key1: value1, key2: value2}.
```

Note that the key value pairs are separated by one colon and the pairs are divided by comas, all of which are contained in a pair of curly braces.

Note that key value pairs are not ordered in a dictionary in any way. If you want a specific order, you must sort it yourself before using it. The dictionaries you can use are dict type instances/objects.

Example (save as dict.py):

```
# 'ab' is short for 'address-book'
ab = {'Swaroop': 'swaroop@swaroopch.com', 'Larry': 'larry@wall.org',
      'Matsumoto': 'matz@ruby-lang.org',
      'Spammer': 'spammer@hotmail.com'
     }

print ("Swaroop's address is", ab['Swaroop'])
# Deleting a key-value pair
del ab['Spammer']

print ('\nThere are {} contacts in the address-book\n'.format
      (len (ab)))

for name, address in ab.items ():
    print ('Contact {} at {}'.format (name, address))
# Adding a key-value pair
ab['Guido'] = 'guido@python.org'
if 'Guido' in ab:
    print ("\nGuido's address is", ab['Guido'])
```

Output:

```
$ python ds_using_dict.py
Swaroop's address is swaroop@swaroopch.com
There are 3 contacts in the address-book
Contact Swaroop at swaroop@swaroopch.com
Contact Matsumoto at matz@ruby-lang.org
Contact Larry at larry@wall.org
Guido's address is guido@python.org
```

- **SET:** Sets of simple items are unordered arrays. They are used where something is more important than the order or how much the presence of an item in a set is.

You can use sets to check membership if it is a subset of another set, find the crossing between two sets, etc.

For example (run it in python interpreter or in IDE):

```
>>>bri = set (['brazil', 'russia', 'india'])
>>>'india' in bri
True
>>>'usa' in bri
False
>>>bric = bri.copy ()
```



```
>>>bric.add ('china')
>>>bric.issuperset (bri)
True
>>>bri.remove ('russia')
>>>bri & bric # OR bri.intersection (bric)
{'brazil', 'india'}
```

OBJECT ORIENTED PROGRAMMING

We have built our program for all the programs which we have written so far in order to manipulate data, i.e., blocks of statements. It is known as the process-oriented programming task. Another way to organize the program is to combine data and features and bundle them in something that is called an entity. The programming style is called object orientated. You may typically use procedural programming, but you can use object-orientated programming techniques when writing large programs or if there is a problem which is better suited to this process.

The two key aspects of object-oriented programming are classes and objects. A class provides a different kind in which objects are class instances. An analogy is that you can have int variables that indicate that the integer saving variables are int class instances (objects).

Objects can store data using standard object variables. Fields are referred to as attribute belonging to an object or class. Objects may also provide functions through the use of class-owned functions. These functions are known as class methods. This terminology is important since it allows us to distinguish between independent and class or object-specific functions and variables. The fields and methods may collectively be referred to as the class attributes.

Fields are two-type-they can be categorized into any instance/object or they can be categorized by themselves. They are respectively referred to as variables of instance and class.

Using the keyword class is developed. The class fields and methods are shown in a series of indents.

The SELF: There is only a single particular difference to the class methods from ordinary functions-they require an additional first name to be applied to the start of the parameter list, but when you call a method, Python doesn't give this a meaning. This attribute refers to the object itself and is called self by default.

While it is highly advised that you use the self name for this parameter-it is undoubtedly conceivable that any other name is frowned upon. There are several benefits of using the common name-any user of your software can know it instantly, and even advanced IDEs.

You have to wonder how Python gives self meaning and why you should not give it meaning. This is made clear by an illustration. Say you have the MyClass class and a myObject class case. When you call a myobject.method (arg1,arg2) method this is translated by Python automatically into MyClass.method (myobject, arg1,arg2) –

that's about the whole thing. This is a unique self.

This also implies even if you have an argument-free process, there is always an argument-the "self."

CLASSES:

The simplest class possible is shown in the following example (save as oop_simplestclass.py).

```
class Person:
    pass # An empty block
p = Person ()
print (p)
```

Output:

```
$ python oop_simplestclass.py
<__main__.Person instance at 0x10171f518>
```

Here we see the self in action. Notice that the say_hi method takes no parameters but still has the self in the function definition.

The __init__ Method:

In the Python classes there are several method names that have a specific relevance. The sense of the init process is now clear.

If an object of a class is instantiated (e.g., created), the init method is executed.

The method is useful for any initialization you want to create (i.e., pass initial values to your object). Please note the double focus at the start and the finish of the term.

Example (save as oop_init.py):

```
class Person:
    def __init__ (self, name):
        self.name = name
def say_hi (self):
    print ('Hello, my name is', self.name)
p = Person ('Mahesh')
p.say_hi ()
# The previous 2 lines can also be written as
# Person ('Swaroop').say_hi ()
```

Output:

```
$ python oop_init.py
Hello, my name is Mahesh
```

Here the init approach is specified as the name of a parameter (including the popular self). We're just creating a new area named name here. Remember that there are two variables as both are named 'name.' Both are distinct. The dotted notation aut. name indicates that something named "name" is part of the object "self," and a local variable is the other name. There is no ambiguity as we specifically state the name to which we refer.

When you construct new instance p, of the person class, we use the name of the class and then the arguments in the parentheses: p = Person ('Mahesh').

We do not explicitly call the `__init__` method. This is the special significance of this method.

Now, we are able to use the `self.name` field in our methods which is demonstrated in the `say_hi` method.

CLASS and OBJECT VARIABLES

The functionalities of classes and objects (i.e., methods) have been addressed already, and we can now hear about the data component. The data portion, i.e., fields, are just ordinary variables that are connected to class and object name spaces. It means that these names only refer to these classes and objects. That's why name spaces are named.

Two types of fields are classified according to whether the class or object is the property of the variables respectively

Class variables are shared-all instances of that class can access them. There is only 1 copy of the class variable, which will be used by all other instances if an object transitions to a class variable.

Every class object/instance is held by the object variables. In this case, each object has its own copy of the field, i.e., in a separate instance it is not shared and in no way is connected to the field by the same name. This will be easy to understand by an example (save as `oop_objvar.py`):

```
class Robot:
    """Represents a robot, with a name."""
    # A class variable, counting the number of robots
    population = 0
    def __init__(self, name):
        """Initializes the data."""
        self.name = name
        print (" (Initializing {})".format (self.name))
        # When this person is created, the robot
        # adds to the population
        Robot.population += 1
    def die (self):
        """I am dying."""
```

```
print ("{} is being destroyed!".format (self.name))
Robot.population -= 1
if Robot.population == 0:
    print ("{} was the last one.".format (self.name))
else:
    print ("There are still {:d} robots working.".format
          (Robot.population))

def say_hi (self):
    """Greeting by the robot.
    Yeah, they can do that."""
    print ("Greetings, my masters call me {}".format
          (self.name))

    @classmethod
    def how_many (cls):
        """Prints the current population."""
        print ("We have {:d} robots.".format (cls.population))

droid1 = Robot ("R2-D2")
droid1.say_hi ()
Robot.how_many ()
droid2 = Robot ("C-3PO")
droid2.say_hi ()
Robot.how_many ()
print ("\nRobots can do some work here.\n")
print ("Robots have finished their work. So let's destroy them.")
droid1.die ()
droid2.die ()
Robot.how_many ()
```

Output:

```
$ python oop_objvar.py
(Initializing R2-D2)
Greetings, my masters call me R2-D2.
We have 1 robots.
(Initializing C-3PO)
Greetings, my masters call me C-3PO.
We have 2 robots.
```

```
Robots can do some work here.  
Robots have finished their work. So let's destroy them.  
R2-D2 is being destroyed!  
There are still 1 robots working.  
C-3PO is being destroyed!  
C-3PO was the last one.  
We have 0 robots.
```

It is a broad example of how class and object variables are to be illustrated. In this case, the population belongs to the Robot class and is thus a class attribute. The name variable is an object variable (the variable is allocated by itself).

We also refer to the population variable as Robot.population rather than Self.population. We use self.name notation in the object's methods to refer to the object variable name. Notice this basic difference between the variables of class and object. Note also that the variable value would be obscured by an object of the same name as a value!

We should have been using self.class.population instead of Robot.population, because any object is referenced to its class through the self.class attribute.

How many is a function that does not belong to the object but rather to the class. This means that, depending on whether we need to learn which class we are in, we can describe it as either a dynamic method or a static one. We use the class form since we refer to a class attribute.

We have listed the many ways of using a decorator as a class tool.

Decorators can be thought of as shortcuts to a function called by the wrapper (e.g., a function that “wraps” around a function so it can do something before or after the internal operation). Thus, using the @classmethod decorator is the same as calling:

how many = classmethod (how many).

In this way, we increase the population number by 1 as we add another robot. Remember that self.name values are unique to each entity that shows the existence of the variables of the entity.

Remember that the variables and methods of the same object should be referred to using only the self. This is regarded as a reference attribute.

We also see in this software the use of class and process docstrings. We can use Robot.doc and the docstring method Robot.say hi.doc In the die method, we simply lower the Robot.population count by 1. In the die method, we may access a docstring class.

Each member of the class is public. One exception is that Python uses name-mangling to effectively render a private variable by means of names that use the double-underline prefix like private var.

The rule is that every variable to use within the class or object will start with the underscore, and all other names are public and can be used for all classes or objects.

This practice is followed. Recall that this is just a convention and not enforced by Python (except a prefix with double emphasis).

INHERITANCE

Reuse of code is one of the main benefits of object-orientated programming and one way to accomplish this is through the heritage mechanism. Inheritance can best be interpreted as a relationship between classes of type and subtype.

Suppose you want to write a program that tracks teachers and students in a university. They have some common features including name, age and address.

These have unique features, such as wage, courses and instructor fees and student charges.

You can build and process two independent classes for each type, but adding a common feature will add to these two independent classes. It's quickly becoming unmanageable.

This will be a great way to build a single class called Schoolmember and then have the instructor and student classes belonging to this class and then we can apply different features to those subtypes.

This strategy has several advantages. If in SchoolMember, we add/modify a function this is reflected automatically in the subtypes. For example, by simply adding it to the SchoolMember class, you can create a new ID card area for both teachers and students. But changes in the subtypes have no effect on other subtypes.

One benefit is that you can refer to a teacher or student object as an object SchoolMember that can be helpful in some circumstances such as the number of schoolmembers counting. This is called polymorphism where a sub-type can be substituted in any situation where a parent type is expected, i.e., the object can be treated as an instance of the parent class.

Notice that, in the case of separate classes, we reuse the class code and do not have to replicate it in various classes like we had to do.

In this case, the Schoolmember class is called the Main or Superclass class. The classes of teachers and students are called the derived grades.

For Example (save as inheritace.py):

```
class SchoolMember:
    '''Represents any school member.'''
    def __init__(self, name, age):
        self.name = name
        self.age = age
    print ('(Initialized SchoolMember: {})' .format (self.name))
def tell (self):
    '''Tell my details.'''
    print ('Name:{}'.format (self.name),
        end=" ")
```

```
class Teacher (SchoolMember):
    '''Represents a teacher.'''
    def __init__ (self, name, age, salary):
        SchoolMember.__init__ (self, name, age)
        self.salary = salary
        print ('(Initialized Teacher: {})' .format (self.name))
    def tell (self):
        SchoolMember.tell (self)
        print ('Salary: "{:d}"' .format (self.salary))

class Student (SchoolMember):
    '''Represents a student.'''
    def __init__ (self, name, age, marks):
        SchoolMember.__init__ (self, name, age)
        self.marks = marks
        print ('(Initialized Student: {})' .format (self.name))
    def tell (self):
        SchoolMember.tell (self)
        print ('Marks: "{:d}"' .format (self.marks))

t = Teacher ('Mrs. Shrividya', 40, 30000)
s = Student ('Swaroop', 25, 75)
# prints a blank line
print ()
members = [t, s]
for member in members:
    # Works for both Teachers and Students
    member.tell ()
```

Output:

```
$ python oop_subclass.py
(Initialized SchoolMember: Mrs. Shrividya)
(Initialized Teacher: Mrs. Shrividya)
(Initialized SchoolMember: Swaroop)
(Initialized Student: Swaroop)
Name:"Mrs. Shrividya" Age:"40" Salary: "30000"
Name:"Swaroop" Age:"25" Marks: "75"
```

FILES: By creating an object in the class of files and using the file's reading, reading or writing methods, you can open and use the file to read or write. It depends on the

mode you have defined for file opening to read or write the file. Then you eventually call the close method when you finish the file to inform Python that we're finished using the script.

Example (save as file.py):

```
poem = '''\
Programming is fun
When the work is done if you wanna make your work also fun: use
Python!'''
# Open for 'w'riting
f = open ('poem.txt', 'w')
# Write text to file
f.write (poem)
# Close the file
f.close ()
# If no mode is specified,
# 'r'ead mode is assumed by default
f = open ('poem.txt')
while True:
    line = f.readline ()
    # Zero length indicates EOF
    if len (line) == 0:
        break
        # The 'line' already has a newline
# at the end of each line
# since it is reading from a file.
print (line, end="")
# close the file
f.close ()
```

Output:

```
$ python3 io_using_file.py
Programming is fun
When the work is done
if you wanna make your work also fun:
use Python
```


Notice that by simply using open method we can create a new file object. Through using the built-in open feature and specifying a file name and the mode where we want to open the file, we open (or build it if it does not already exist). Mode can include the read ('r') mode, write ('w') or add ('a') mode. We may also indicate, in the text ('t') or in the binary ('b'), if you are reading, writing, or appending. In reality, several more modes are available and help (open) will give you more information.

Open) (detects the file by default as a 'text file and then opens it in' read mode.

In our example, we initially open/create the file in text mode and use the file object writing procedure for writing our variable string poem and then close the file.

First, for viewing, we open the same file again. We do not have to specify a mode because the default mode is 'read text file.' We read the readline method in a loop in each line of the file. The entire line with the newline character at the end of the line is restored. When we return a blank string, it means we are at the end of the file and break out of the loop.

In the end, we finally close the file.

We can see from our readline output that this program has indeed written to and read from our new poem.txt file.

Summary

As we have discussed the important basics of python and every concept of python programming language is discussed with suitable examples. Python will be used in further chapter implementation parts and for making real time projects which can be used in real time for research and academic studies. In the implementation of python for further branches of artificial Intelligence there are several Libraries and frameworks are involved which will be discussed from topic to topic with its installation and uses with different algorithms.